Validate and Rainbow speaker study guide
- Slides 22 and 23 validate a controller
- Slides 24 and 25 connect the examples
- Slides 26 and 27 introduce Rainbow
- Slides 28 to 30 explain Bellman contraction
- Slides 31 to 33 explain the plant work
- Questions to practise aloud
- Run the examples and inspect the evidence

Validate and Rainbow speaker study guide

The Validate section executes a controller against every event sequence represented by a finite model. The Rainbow section checks mathematical proof terms. These are different forms of evidence. Neither automatically proves that a physical turbine is safe or that the browser implements an exact mathematical theorem.

Read the controller example first, then the small composition proof, then Bellman contraction. Finally read the plant example. Each section gives you a sentence you can say on stage, the mechanism behind it, and the boundary of the claim.

The slide numbers refer to the current 36-slide deck. This is a study companion; the shorter spoken script remains in `SCRIPT.md`.

Slides 22 and 23 validate a controller

Say: “We have a controller and a rule. Can any allowed sequence of events make the controller break the rule?”

The earlier optimization chose tomorrow’s turbine settings. Here we use a separate, deliberately small controller example. We check its RUN or STOP commands after sensor events. We do not feed the 72-setting optimized schedule into this validator.

The rule is: a confident detection must produce a STOP command in the same modeled minute. This is a requirement on software output, not a statement about braking time or sensor accuracy.

The state and the event

```
;; Controller state before a new detection
{:ctx :unseen :det 0 :u 0.3
 :action :run :minute 1 :waiting nil}

;; Incoming event
{:type :detect :level 2}
```

Field	Meaning
`:ctx`	Context, simplified to known (`:seen`) or unknown (`:unseen`) to the table. It does not mean whether a bird was seen.
`:det`	Strongest detection recorded during the current ten-minute slot: 0 none, 1 weak, 2 confident.
`:level`	Strength reported by the incoming detection event. The controller combines this with the stored `:det`.
`:u`	Random draw used by the probabilistic policy, held constant during a slot. It is not detection confidence.
`:action`	The controller's current command, `:run` or `:stop`.
`:minute`	Position in the modeled ten-minute slot. This lets the search distinguish timing cases.
`:waiting`	Bookkeeping for how long confident detection has remained unanswered. It does not choose the action.

The event handler uses `max` to retain the strongest detection. A weak event after a confident one does not erase the confident detection. A timer tick resets the recorded level after making its decision.

The table is a policy input

This is the actual example table in `talk.core`:

```
(def stop-table
  {:seen {0 0.05, 1 0.6, 2 1.0}})
```

Context	No detection	Weak detection	Confident detection
Known	5% stop probability	60%	100%
Unknown	Missing	Missing	Missing

It is a hard-coded stand-in for a learned decision table. There is no training procedure in the slide, and these probabilities are illustrative.

`table-decision` accepts this table and returns a function from state to command:

```
(def decide-by-table (table-decision stop-table))

(decide-by-table {:ctx :seen :det 2 :u 0.3})
;; => :stop

(decide-by-table {:ctx :unseen :det 2 :u 0.3})
;; => :run
```

Inside it, `(get-in table [ctx det] 0)` reads the nested entry and defaults to zero if it is missing. For the known confident case it compares `0.3 < 1`; for the unknown case it compares `0.3 < 0`. That fallback silently treats an unknown situation as "never stop."

For a probability of 0.6, a uniform draw below 0.6 chooses STOP. The finite search uses three representative draws, 0.02, 0.3 and 0.8. They cover the decision branches for this table's thresholds. The search enumerates those choices; it does not randomly sample trajectories. Changing the table's thresholds can require changing that abstraction.

The two wrappers solve different problems

The original `tick-controller` records detections as they arrive but chooses an action only at a timer tick. A confident detection just after a tick can therefore leave the command at RUN.

`react-to-detections` wraps that controller. It first calls the original controller to update the state. If the event is a detection and the recorded level increased, it calls the supplied decision function immediately and puts its result in `:action`. Equal or weaker detections do not trigger another decision.

This fixes the delay. It does not fix the table's unknown-context fallback. An immediate decision can still return RUN.

`with-shield` wraps the decision function. For a context present in the table, it delegates to the original decision. For an absent context, any positive detection produces STOP; no detection produces RUN.

```
(def shielded-decision
  (with-shield decide-by-table stop-table))

(def controller
  (react-to-detections
    (tick-controller shielded-decision)
    shielded-decision))
```

The shielded decision is supplied twice because there are two occasions to decide: timer ticks and stronger detections. Both must use the same fallback rule.

The Clojure syntax is ordinary destructuring. `{:keys [ctx det] :as s}` extracts `ctx` and `det` while retaining the whole input map as `s`. `assoc` returns a new map with a changed field; it does not mutate the previous state.

Follow one complete call

For the state and confident event above, all three controllers record `:det 2`:

Controller	What happens next	Resulting command
Timer only	Records the detection and waits for the timer	RUN
Reacting	Calls the table immediately; the context is missing	RUN
Reacting with shield	Calls the shield; unknown context plus detection stops	STOP

The full call flow is:

```
validator generates a possible event
  → composed controller receives state and event
    → base controller records the detection
    → reaction wrapper notices that it became stronger
      → shielded decision reads the updated state
        → known context: table lookup and draw comparison
        → unknown context: STOP if detection is positive
    → new state contains the resulting action
  → validator checks the new state for a rule violation
```

The validator does not invent an action. It executes the controller and inspects its output.

The shield has a specific boundary

It only checks whether the context exists. It does not inspect the quality of a known row or repair a missing detection entry within a known row.

For example, replacing the known row with `{0 0, 1 0, 2 0}` makes a confident known detection return RUN even with this shield. The study runner demonstrates that case. You can say the shield repairs this example's unknown-context failure. You cannot say it makes an arbitrary controller safe.

Reachability is where iterate enters

Say: "Each update adds every state we could reach next. We keep updating until the set stops growing."

```
(defn reach-step [successors]
  (fn [states]
    (into states (mapcat successors states))))

(defn reachable [successors initial]
  (fixed-point (reach-step successors)
               = (set initial)))
```

`successors` returns possible next states from one state. `mapcat` collects those results across the current set. `into` adds them to the old set, retaining everything already reached and removing duplicates because the destination is a set.

With a tiny cyclic graph `a → b → c → a`, the sequence is:

```
#{:a}
#{:a :b}
#{:a :b :c}
#{:a :b :c}  ; equality: closed under the successor function
```

Cycles do not require exploring infinitely many distinct histories. Once every successor is already in the set, further expansion cannot add anything. This depends on the state containing the information needed to determine future behavior. Here it includes detection, context, random-draw band, action and timing.

The controller model branches on no detection, a weak detection, or a confident detection in each minute. At the slot boundary it explores timer ticks with the modeled contexts and draw bands. It records the detection state before the next timer reset, so a reset cannot erase a violation before inspection.

The bad-state predicate is small:

```
(defn unanswered? [{:keys [det action]}]
  (and (= 2 det) (= :run action)))
```

Current results from the deck's actual functions:

Controller	Reachable states	States breaking the rule
Timer only	473	275
Reacting	343	165
Reacting with shield	241	0

These are counts of distinct states, not probabilities, incidents, or birds. All three searches reach a closed set. The shielded result covers every path represented by this finite transition model. It does not validate bird recognition, physical braking, the optimized day schedule, or unmodeled events.

The fixed-point helper has a work cap. In general, reaching that cap would not establish closure. The study runner explicitly checks that applying the expansion once more adds no state.

Slides 24 and 25 connect the examples

Say: "Underneath each example is iterate. We changed the state and the function it repeatedly applies."

Example	State	Update	Stopping test
Optimization	A vector of turbine settings	A gradient-based move	Settings change little
Decision-making	A table of expected future profits	Bellman backup	Values change little

Example	State	Update	Stopping test
Validation	A set of reachable controller states	Add every successor	Exact set equality

Small numerical movement and exact set closure are different conditions. `iterate` does not supply the argument that a stopping condition proves what we want. We supply that argument for each problem.

`with-observer` is another wrapper: it calls a step, reports old and new states to an observer, and returns the new state. The displayed example realizes five states, including the initial state, so it logs four updates. The logging effect occurs when the lazy sequence is consumed.

Slides 26 and 27 introduce Rainbow

Say: "Rainbow is a theorem prover written in Clojure. Proofs are data, and a kernel checks whether a proposed term establishes its stated theorem."

A kernel is the small program implementing the logical checking rules. Proof search and tactics may propose a term; the kernel is responsible for accepting or rejecting it. Choosing Clojure does not itself supply proofs. You must state the property and construct evidence for it. Static typing and theorem proving provide different guarantees and can be used together.

Read the exact proof syntax

This is the source shown on slide 27:

```
{:type
 [:forall [[P Prop] [Q Prop] [R Prop]]
  [:-> [:-> P Q] [:-> Q R] P R]]

 :value
 [:fn [[P Prop] [Q Prop] [R Prop]
       [f [:-> P Q]] [g [:-> Q R]] [p P]]
  [:app g [:app f p]]]}
```

The outer `:type` and `:value` fields are the input format used by our checker script: the claimed type and the proposed term. The vectors inside are Rainbow's explicit term syntax, represented as EDN. They are not calls to Clojure's `fn` or `->` macro.

Syntax	Read it aloud
`Prop`	The sort of propositions

Syntax	Read it aloud
`[:forall [[P Prop] ...] body]`	For arbitrary propositions P, Q and R, establish this body
`[:-> P Q]`	A proof of P can be turned into a proof of Q
`[f [:-> P Q]]`	f is such a proof-transforming function
`[p P]`	p is a proof of P
`[:fn [bindings] body]`	Bind the arguments and construct the proof body
`[:app f p]`	Apply f to p; the result proves Q
`[:app g [:app f p]]`	Apply g to that Q proof; the result proves R

The whole theorem says: for any P, Q and R, given `f : P → Q`, `g : Q → R` and `p : P`, produce a proof of R. The multi-argument arrow is right-associated: `(P → Q) → (Q → R) → P → R`.

The proof is composition:

```
p proves P
f(p) proves Q
g(f(p)) proves R
```

This does not establish P, Q or R unconditionally. It establishes the implication with those inputs. The universally quantified statement works for every choice of propositions with the required proof arguments.

Why the wrong proof is rejected

Replace the body with `[:app f p]`. It produces a proof of Q, but the expected result is a proof of R. P, Q and R are arbitrary; the kernel cannot assume Q equals R. The term is rejected.

The tiny composition proof and this negative control were checked again on 23 October 2026 using the clean Rainbow revision `d9080c7285c7cd28a81e2d184b25ecae333510fd`. The browser shows the recorded verdict; it does not execute a Rainbow kernel.

“No axioms” means no additional axiom declarations were required by this proof. It does not eliminate trust in the kernel, runtime or logical rules. The argument parameters f, g and p are explicit assumptions inside the implication, not secretly introduced global axioms.

Edit the proof as data (slide 27, third phase)

Say: “The editor proposes a new term. The kernel checks it. We can change the interface without changing what counts as a proof.”

The slide now has three phases: the original composition, removal of the outer application, and insertion of that application again. Its tree is drawn from the current term. It is not a separately maintained illustration.

The body lives at `[:value 2]`: the map’s `:value` is a `[:fn bindings body]` vector, whose zero-based element 2 is the body. These are ordinary Clojure functions from `src/talk/proof_data.cljc`:

```
(defn remove-outer [proof]
  (update-in proof [:value 2]
             (fn [body] (nth body 2))))

(defn wrap-body [proof function-name]
  (update-in proof [:value 2]
             (fn [body] [:app function-name body])))

(def shorter (remove-outer proof))
(wrap-body shorter 'g)
```

`update-in` is the higher-order function: it receives a function to apply at the selected location and returns a new data structure. `remove-outer` selects the argument from the outer `[:app g argument]` vector. Here that argument is `[:app f p]`. `wrap-body` constructs `[:app g [:app f p]]` again. The quoted `g` is a symbol in a proof term; it is not a Clojure function being called. The original proof remains available.

The shared functions run both in the browser and in the JVM check script. The script checks the original, removed and restored terms; the browser asserts that its edits produce exactly those recorded terms before displaying their verdicts. Rechecked on 23 October against the same clean `d9080c7` revision: checked, rejected, checked. No kernel executes in the browser.

This is a small demonstration for this application-shaped term, not a general proof editor. Arbitrary edits need to respect binders, variable scope, dependencies and the claimed type. A well-formed EDN vector can still be an invalid proof. The record also does not establish that every part of the kernel is sound.

Why this matters beyond one edit

Clojure forms have a data representation. Rainbow explicit proof terms use vectors and symbols with their own grammar. They are not the same language, but ordinary Clojure functions can manipulate both representations. That is the connection to homoiconicity; choosing Clojure alone does not make a proof correct.

A visual tool could select a subtree, insert a step, show a dependency, or propose an edit from a drawing gesture. Each creates a candidate term requiring a check. Editing means producing a new immutable value, not modifying the accepted artifact in place. The verdict on an old term does not certify a changed term.

On slide 34, “Where I want to take Rainbow,” describe the broader OR tools as future work. The small tree/edit demo exists in this talk. A general visual editor, certificate tree replay and assumption-oriented OR views are not demonstrated as released Rainbow products. Visualization is not exclusive to Rainbow or Clojure; the opportunity is an interface grounded in the actual proof data and the operator’s questions.

Slides 28 to 30 explain Bellman contraction

Say: “Suppose two tables disagree about future profit. Applying the same Bellman update brings their largest disagreement to at most ninety percent of its previous size.”

Our discount factor is 0.9. This bounds the gap between two tables; it does not mean every individual table entry decreases.

Let `B` be `(bellman restock)`. V and W are two estimates for the same model. Define their distance as the largest absolute difference across states:

```
distance(V, W) = max over states s of |V(s) - W(s)|

distance(B(V), B(W)) ≤ γ × distance(V, W)
```

Our example uses γ = 0.9. One table is zero everywhere and the other is ten everywhere. Every action’s expected future term shifts by nine because the demand probabilities sum to one. The maximum therefore shifts by nine too. Browser arithmetic produces approximately 9.000000000000004, displayed as 9. This numerical example is not a universal proof.

The proof follows the operations in q

Recall that q scores an order using expected immediate profit plus discounted future value. Write `Q_V(s,a)` for that score when the future table is V.

Assume every entry of V and W differs by at most ε. Fix the same state s and the same action a in both calculations.

1. **Subtract the scores.** The immediate reward is the same in both, so it cancels. Only the future-value terms differ.
2. **Bound the weighted sum.** Each future difference has absolute value at most ε. The probabilities are nonnegative and sum to one. Thus the score difference is at most $\Sigma\ p \times \gamma \times \varepsilon = \gamma \times \varepsilon$.
3. **Take the best action.** For every a, $Q_V(s,a) \le Q_W(s,a) + \gamma\ \varepsilon \le \max_b\ Q_W(s,b) + \gamma\ \varepsilon$. Taking the maximum on the left gives $B(V)(s) \le B(W)(s) + \gamma\ \varepsilon$.
4. **Reverse V and W.** The same argument gives the other inequality. Together they bound the absolute difference, for every state.

The winning action is allowed to change. The proof does not assume that V and W recommend the same order. This is the subtle reason the `max` step is valid.

The finite model must have nonempty available-action sets, normalized nonnegative probabilities, and the same rewards/transitions in both evaluations. Strict contraction needs $0 \le \gamma < 1$. With $\gamma = 1$ this argument gives a nonexpansion bound, not the contraction used for the residual result below.

What the residual says

For an exact candidate table V, let $\delta = \text{distance}(V, B(V))$. If V* is the Bellman fixed point, the triangle inequality and contraction give:

```
distance(V, V*)
  ≤ distance(V, B(V)) + distance(B(V), B(V*))
  ≤ δ + γ × distance(V, V*)

distance(V, V*) ≤ δ / (1 − γ)
```

With $\gamma = 0.9$, an exact residual of 0.001 bounds value error by 0.01. This bounds value estimates; it does not automatically prove that a greedy order is uniquely optimal. Close action scores can change the chosen action.

The residual compares a candidate with one more update. A change between two consecutive iterates is the residual of the earlier table. A floating-point stopping test without a rounding-error analysis is not an exact residual certificate.

What the talk can claim about the formal proof

The composition example has a fresh kernel check. The Bellman theorem has a historical receipt dated 9 October 2026, named `value/signed-optimality-step-additively-contractive`, with an empty recorded axiom profile.

That Bellman receipt predates a documented kernel repair. The kernel's trusted-base document says an earlier bug could accept a proof of False and requires affected artifacts to be replayed after repair. This talk does not have a completed repaired-kernel replay of the Bellman artifact. Slide 30 correctly labels it historical and pending replay.

Say: "The contraction argument is the mathematics we want checked. This is a historical Rainbow receipt; replay after the kernel repair is pending. The browser's floating-point implementation is not proved equivalent to that exact operator."

Do not describe the Bellman browser demo as certified by the theorem. Nor should a theorem that merely unfolds an update definition be described as proving its convergence or optimality.

Slides 31 to 33 explain the plant work

The physical interpretation of the model

An illustrative methanol model uses electricity to make hydrogen, stores hydrogen in a tank, and feeds a methanol loop. Scheduling chooses when equipment runs and when the tank absorbs the difference. Electricity prices vary hourly; minimum loads, starts and a weekly production requirement constrain those choices.

The benchmark uses Spanish OMIE wholesale prices known for the whole week. It is not a named operating factory, a weather model, a live trading strategy, or a renewable-electricity supply contract. The modeled sizes are 120 MW of electrolysis, 20 tonnes of hydrogen storage and a methanol loop capable of 10 tonnes per hour, with a 588-tonne weekly requirement.

Who performs each calculation

Say: “Our Clojure code builds the model. HiGHS finds a proposed schedule. Our search builds a certificate using HiGHS for its smaller optimization problems. Rainbow checks that certificate.”

Component	Role
Our model	Encodes equipment, storage, prices and production constraints
HiGHS	Third-party solver; finds the candidate schedule and solves LP relaxations inside our search
Our branch-and-bound producer	Chooses branches and constructs the tree and exact bound evidence
Rainbow checker	Checks the submitted certificate using exact arithmetic

An LP relaxation allows integer decisions to be fractional. For minimization, its optimum is a lower bound on the corresponding integer problem. Branching partitions integer possibilities; exact dual evidence supplies bounds or demonstrates that a branch is infeasible. The checker accounts for the branches instead of relying solely on the solver’s “optimal” status.

What one part per million means

For a minimization problem, let U be the cost of a feasible proposed schedule and L a lower bound on all feasible schedules. Then $L \le \text{optimum} \le U$; the distance still available for improvement is at most $U - L$.

The reported weekly certificate results cover 52 weeks of 2023 at relative tolerance 10^{-6}. The largest reported absolute gap is approximately €0.26 on a week costing approximately €388,000. This is not the amount saved against a baseline. It is the remaining improvement permitted by the reported bound. A nonzero gap permits a slightly cheaper schedule to exist.

The important unfinished piece is the checker's **soundness theorem**: a formal proof that acceptance of any certificate implies the claimed optimality bound. The checker program and exact checks exist, but that general implication remains open. The small composition proof, the historical Bellman receipt and this checker-soundness question are three separate things.

Say: "The reported certificates were accepted by our exact checker. The general theorem connecting checker acceptance to optimality is still open."

Avoid "I have a complete formal proof that the real plant is always within 10^{-6} of optimal." That would combine an unfinished formal link, a finite benchmark, a modeled plant and an unmodeled real operating environment.

The San Roque savings slide is a separate study

Slide 33 concerns an assumed e-SAF design near San Roque / Los Barrios, at 36.18° N, 5.40° W. It uses local wind and solar data. The modeled mean saving of €4.4m/year concerns energy and backup hydrogen versus the best tuned operating rule; the reported interval is €2.5–6.3m.

These are modeled savings, not measured customer savings. The nine cases combine weather and price years; price years were also used in tuning, so they are not nine independent future years. The methanol certificate result does not certify this separate e-SAF comparison.

Questions to practise aloud

Try each answer without looking at the earlier sections. A short accurate answer is enough.

1. What is the difference between `:det`, `:level` and `:u`?
2. Where does the STOP/RUN action come from? Does the validator choose it?
3. Why are both wrappers needed? What case can this shield still miss?
4. Why does set equality let the reachability search stop, even with cycles?
5. Does zero bad states mean no real bird can be harmed?
6. Read `[:app g [:app f p]]` and explain the rejected alternative.
7. Does an empty axiom list remove the need to trust anything?
8. What cancels in the Bellman proof? Why can the maximizing action change?
9. Does a larger value table necessarily represent a better policy?
10. What does residual 0.001 imply when γ is 0.9, and under what arithmetic assumptions?
11. Which proof is freshly checked, which receipt is historical, and which theorem is open?
12. What does HiGHS do, and what did we implement?
13. How do the €4.4m saving and the €0.26 gap differ?

Answer checks

1. `:det` is recorded detection, `:level` is an incoming report, and `:u` is a held random draw used by the policy.
2. The supplied controller calls its decision function. The validator generates allowed inputs and checks returned states.
3. Reacting fixes timing; shielding fixes unknown-context fallback. A bad known row is still delegated to.
4. The accumulated finite set already contains every successor. The state must retain the information needed for future transitions.
5. No. It establishes the command rule within the encoded model, not sensor correctness or physical safety.
6. f takes p to a Q proof; g takes that to an R proof. Returning only f(p) supplies Q where R is required.
7. No. Logical checking still trusts the kernel and its execution environment.
8. The same immediate reward cancels. Each action's score satisfies the bound, so taking maxima preserves the bound without requiring the same winner.
9. No. Value iteration also extends the planning horizon; a value is a money estimate and a policy is an action choice.
10. An exact candidate's value error is at most 0.01 under the discounted-model assumptions. Floating-point rounding needs separate treatment.
11. Composition: fresh accepted/rejected check. Bellman: historical receipt pending repaired-kernel replay. Plant checker: acceptance-implies-optimality theorem open.
12. HiGHS supplies the candidate and LP solves. Our code supplies the model, certificate-producing search and checking machinery.
13. €4.4m is a separate e-SAF baseline comparison. €0.26 is the largest reported methanol certificate gap, not baseline savings.

Run the examples and inspect the evidence

From the `talk` directory:

```
npm run study
```

This imports the presentation's actual ClojureScript functions. It prints the common detection event and three controller outputs, reachable-state counts, a tiny cyclic fixed point, the shield's bad-known-row counterexample and the numerical Bellman gap. To save the output:

```
npm run study -- speaker/study-results.json
```

From the repository root, check the composition proof and its deliberately wrong alternative:

```
clojure -Sdeps '{:paths ["../rainbow/rainbow/src" "talk/src"]}' \
  -M talk/scripts/check-proof.clj
```

That command requires the local clean Rainbow checkout and updates the talk's composition receipt. It does not replay the Bellman corpus or prove the plant checker's soundness theorem.

The main references are:

- Deck code and controller implementation.
- Runnable study examples and recorded output.
- Exact composition term and fresh check receipt.
- Historical Bellman receipt.
- Plant and savings provenance.
- Rainbow checkout: `../rainbow/rainbow/doc/trusted-base.md` and `doc/claims.md`.
- Plant checker: `../rainbow/internal-milp/src/rainbow/logic/clojure_bb_tree_certificate.clj`.
- Certificate report: `../rainbow/DESIGN-2026-10-18-proved-milp-certificates.md`.

The sibling-repository paths in the last three items are relative to the presentation repository root.